NBA Shot Make Probabilities

Overview

Analytics in the NBA have exploded in the past decade plus, and with that, models for prediction and evaluation have become very important. The goal of this project is to predict the probability a shot is made, based on many factors available from advanced stat tracking. The model I chose was a `RandomForestClassifier` because it captures nonlinear relationships and interactions between this data that a linear model may not do as well. The goal was to predict probabilities, not just class, so you will see I used a built-in RandomForest function to help with this. I also experimented with `LogisticRegression`, to see how a different methodology would compare.

Data

My dataset contained a total of 28 columns, with features such as:

- Shot, shooter, season, offensive/defensive team, and defenders ids.
- Amount and distance of defending contesters.
- Type of shot, and the location on the court.
- Amount of dribbles before the shot, and how fast shooter was moving.
- Time on shotclock.

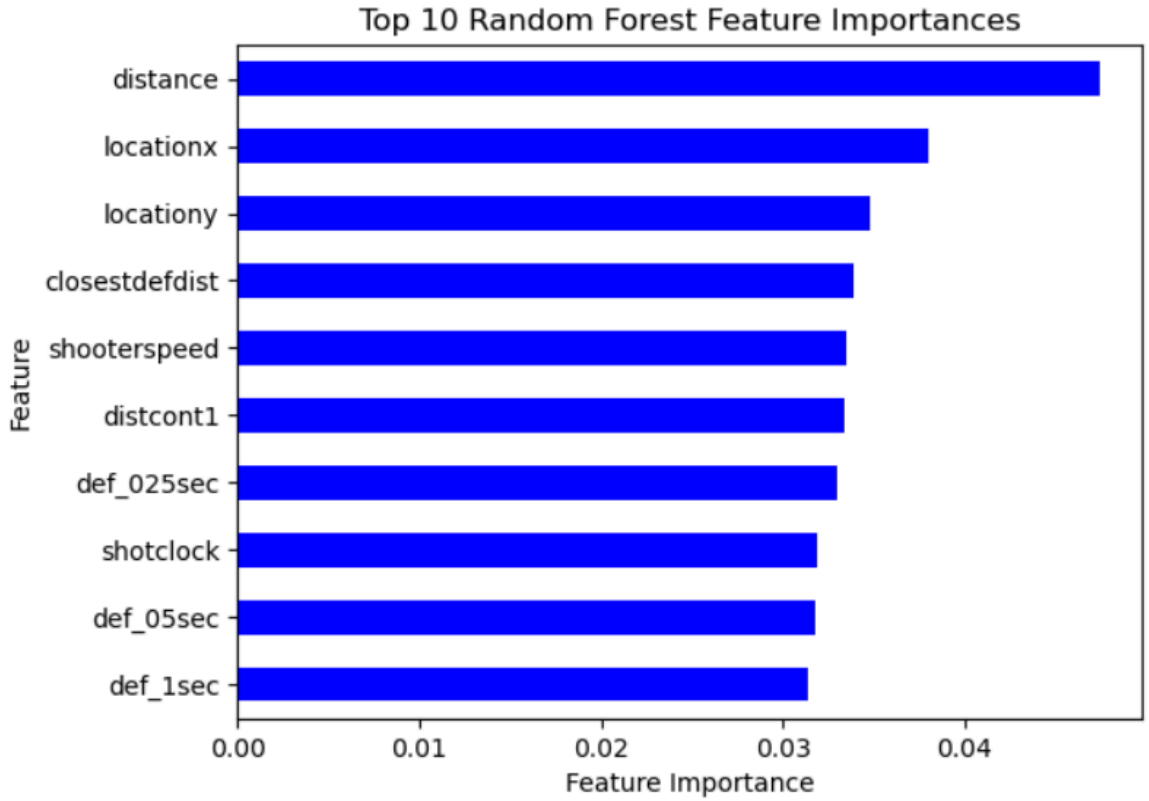
	shot_id	season_id	off_team_id	def_team_id	shooter_id	closestdef_id	num_contesters	contester1_id	contester2_id	contester3_id	...	shottype	distance
0	1	d7a2d	e801cc	16981c	45a36b40c8	c97e433df4	1	c97e433df4	NaN	NaN	...	jumper	25.689301
1	5	d7a2d	b25360	5ab44a	2cb9af8508	9f12db4242	1	9f12db4242	NaN	NaN	...	layup	4.002812
2	6	d7a2d	82f2ae	28049c	90b1c72e19	b25b22c4c2	0	NaN	NaN	NaN	...	jumper	23.003254
3	8	d7a2d	16981c	c3d95c	c40278f00f	eab37a8ca6	1	eab37a8ca6	NaN	NaN	...	floater	9.287723
4	10	d7a2d	08555d	28049c	46a7a7ad66	0e1a8257a8	1	0e1a8257a8	NaN	NaN	...	jumper	25.464332

Features

Initially I did an correlation test, to see what variable might be good predictors for outcome, and as you can see below, there aren't any strong correlations. Strong correlations are generally values of 0.7 or higher, and -0.7 or lower. The strongest here would be: `distance`, `three`, `locationx`, `shooterspeed`, and `contested`. These are only the numerical features, as them being numbers allows for correlations to be calculated. For the model, I decided to include the categorical information (`game state`, `shooter_id`, `shottype`, `off_team_id`, `closestdef_id`, etc.) as well. I converted these categorical variables into one-hot encoded features so that the tree-based models would not interpret player or team IDs as having a meaningful numerical ordering. The hope would be that the model can now learn from every team, contester, and shooter to have better predictions. I decided to keep all features because the relationship between these variables and `outcome` being so small, I figured it would be best to include all data I can.

	outcome
shot_id	0.001511
outcome	1.000000
num_contesters	-0.086399
distcont1	-0.058870
distcont2	0.000114
distcont3	-0.012278
distcont4	0.031627
contested	-0.113747
distance	-0.214814
locationx	-0.157644
locationy	-0.008785
three	-0.164228
dribblesbefore	-0.025048
shotclock	0.085647
closestdefdist	-0.029141
shooterspeed	0.121557
def_1sec	-0.013749
def_075sec	-0.025377
def_05sec	-0.039968
def_025sec	-0.050968

Another reason I included all features is it allowed the model to asses what features were most important. Along with the fact that I now had tons of columns to count as features from one-hot encoding. Including all possible features would lead the model to developing predictors for certain shooters being better than others, certain better defenders, etc. And one-hot encoding allowed me to accomplish this. The results of the `RandomForestClassifier` model runnning it's own feature importance are below:



This ranks features of the model on how well they are at separating shots (data points) into makes and misses. A feature that separates a left branch into 9 makes and 1 miss, and a right branch into 2 makes and 8 misses, will have a high feature importance. As one could guess, where the shooter was when they shot was most important, with the distance and how close their defender closed out to their shot being very important as well.

Model and Output

Through training and validation of both a `RandomForestClassifier` and `LogisticRegression` model, I decided to use my final results from the RandomForest. The RandomForest produced a lower logloss of **0.6308**, compared to LogisticRegression's **0.6334**. This means the RandomForest model was more accurate in predicting correct outcomes.

Random Forests consists of many decision trees trained on different subsets of the data and features. A decision tree's probability is the proportion of training observations belonging to each class in the leaf where the new observation lands. It looks something like:

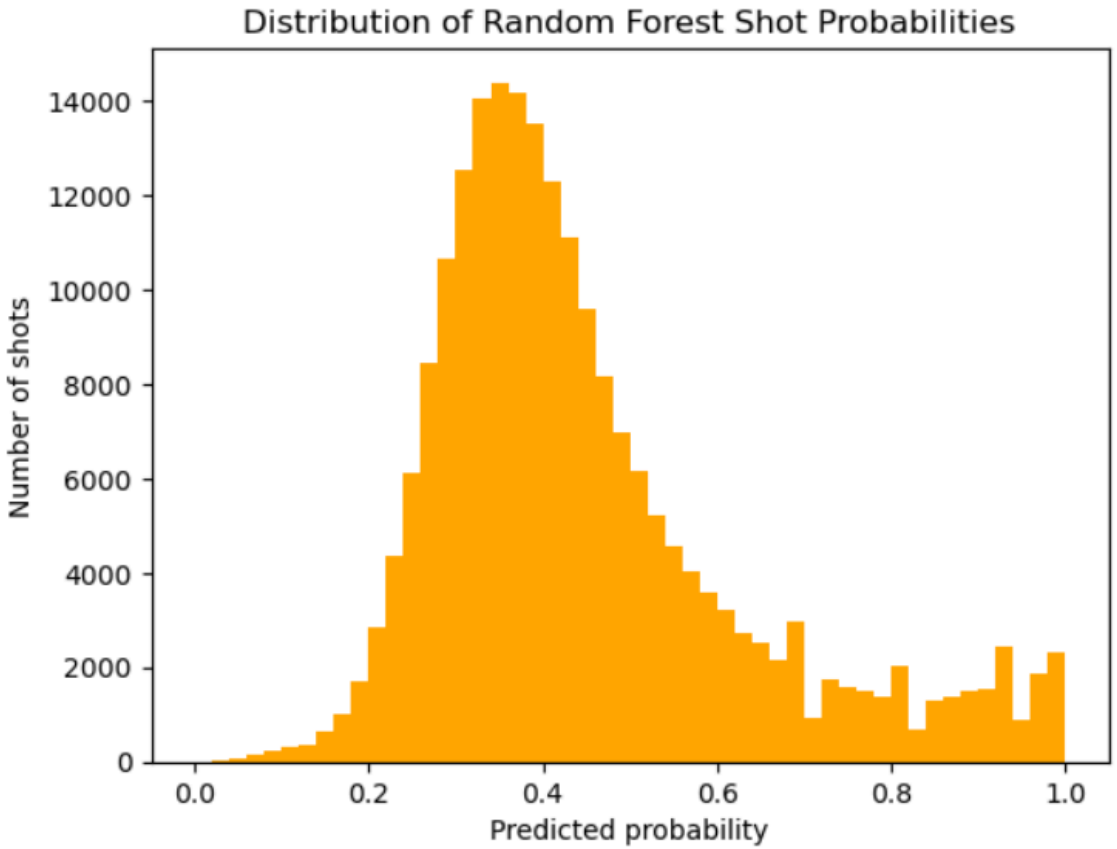
```
Is distance < 15 ft?
|
Yes
|
Is shot type = layup?
|
Yes
|
Leaf B
```

There will be a certain number a data points already stored in this leaf that satisfy these conditions, and the probability of a make is the proportion of these shots. So say 10 shots that fit these conditions are makes, and 10 are misses. The probability for this subset tree is 10/20 = 50%. There will be multiple of this exact tree, but random samples of data points, so the average is then calculated between all of the subset trees for an official probability prediction.

Below are some statistics on my results from the `RandomForestClassifier`. As you can see, the average field goal percentage was predicted to be 44.46%, something I consider reasonable. There isn't an average of 30% percent or something, or crazy population groupings at the extrema that would lead to around 45% average by default. The 50th percentile is right around 40%, which makes sense with more perimeter shots nowadays, a lesser layups, dunks, and closers shots that would have higher percentages. It's interesting it predicts a 100% chance of a shot being made, that's basically impossible! I looked it up, and it's wide open three pointer, probably Steph Curry. Numbers don't lie I guess!

```
count    213977.000000
mean          0.444697
std           0.183522
min           0.000000
25%           0.320000
50%           0.400000
```

```
75%          0.520000
max          1.000000
```



Challenges

I initially did the correlation test to see if it revealed any relationships that didn't need to be included in the model, but with many weak relationships, I decided all features can still be useful when combined with other variables, particularly for a nonlinear model such as a RandomForest.

The biggest challenge I had was handling the categorical features. I wanted my model to have information on shooters, teams, defenders, different type of shots, etc, because these should help for better predictions. And since these were stored as strings, I

had to find a way to convert each string to an usable form of data, and that came about through one-hot encoding. This turned each occurance of every unique team, shooter, defender, month, shottype, and gamestate into hundreds of columns full of 1s and 0s indicating when each unique string would appear for a shot. This created a very high-dimensional dataset and substantially increased model training time.

I also experimented with LogisticRegression, since I know RandomForest trees aren't the most used form of classification. Because of the large number of one-hot encoded columns I now had, I used a sparse representation and the SAGA solver. These are just parameters and solver best fit for large amounts of data. LogisticRegression did still provide a useful comparison because it models the relationship between the features and shot probability linearly, while the RandomForest models nonlinear.

Another issue involved was dealing with NaN values. RandomForests can handle these, but the current version I have for LogisticRegression can't. Columns `distcont1-4` and `contester1-4_id` contain NaNs, and contester was easy to deal with. I was able to one-hot encode those columns, so you would see a column for contester_9eu9r, a column for contester_1234, etc, filled with 0s, and 1s when present. NaNs were tougher to decide on what to do for the distcont columns. I decided to turn all NaNs to 0s, since in the formula for logistic regression, all features will be represented through terms in the equation

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n)}}$$

A distance of a contester being 0 techinicallty means they are directlty on top of the shooter, and "0 feeet away", but mathematically, NaNs being turned into os works here. For example, If distcont2 = 0, then the term inside of the LogisticRegression equation would be B*0 = 0, eliminating that term from the equation; the mathematic way of saying that the distance of the contester is not there. Plus, there is the `num_contesters` column to still indicate how many contester are there for each shot.

Future Improvements

As I've said multiple times, I went through and turned the categorical features into columns for each instance of a team, shooter, defender, shottype, etc, appearing. This added around 3,700 columns to my data, and naturally increased runtime. In the future, I would like to research more efficient ways to handle categorical variables when doing predictions that need numerical data.

Working with a different form of dataframe might benefit me too. I kept running into warnings about sparse columns with my encoded 0s and 1s columns, and there might be dataframes more suited for this work than `pandas`.

As stated earlier, I also created a LogisticRegression model. This had a higher logloss than the RandomForest, but I know LogisticRegression is more standardized, so I could have put more time into that model. Tweaking with parameters in both models could be helpful too, as that is something I didn't have enough time to test on either.

A general improvement could be made on both of my models as well. With logloss for both models being around 0.63, there is definitely still remove for improvement. You won't ever get to a logloss of 0.0, since 100% probability doesn't exactly exist, but there's room to try and get to around 0.50 logloss.

With more time, I would have liked to also explore the results of my predictions. I could do research into what teams have the best shooters, calculate percentages for each player, what defenders have the most effect on lowering shooting percentage, etc.

In [ ]: